

```
# =====  
# STEP 0A – Install BioPython and MAFFT  
# =====  
!pip install biopython -q  
import os, sys  
from Bio import __version__ as bio_ver  
print("BioPython version:", bio_ver)  
!apt-get install -qq -y mafft  
!mafft --version
```

```

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

v7.490 (2021/Oct/30)

```

```

# =====
# STEP 0B – Install RAXML-NG and ModelTest-NG (direct binaries)
# =====

# --- RAXML-NG ---
!wget -q https://github.com/amkozlov/raxml-ng/releases/download/1.2.0/raxml-ng\_v
    -O raxml-ng.zip
!unzip -qo raxml-ng.zip
!cp raxml-ng /usr/local/bin/raxml-ng
!chmod +x /usr/local/bin/raxml-ng

# --- ModelTest-NG (correct static binary) ---
!wget -q \
    "https://github.com/ddarriba/modeltest/releases/download/v0.1.7/modeltest-ng-s
    -O modeltest.tar.gz

import os, tarfile
# Try to extract; fall back to conda if the static binary isn't available
if os.path.getsize("modeltest.tar.gz") > 10000:
    with tarfile.open("modeltest.tar.gz", "r:gz") as tar:
        tar.extractall(".")
    # Locate the binary wherever it landed
    for root, dirs, files in os.walk("."):
        for fn in files:
            if fn == "modeltest-ng":
                src = os.path.join(root, fn)
                !cp {src} /usr/local/bin/modeltest-ng
                !chmod +x /usr/local/bin/modeltest-ng
                print(f"Installed modeltest-ng from {src}")
                break
else:
    print("Static binary not available; trying conda-forge...")
    !pip install -q condacolab
    import condacolab; condacolab.install() # kernel restarts here
    !conda install -y -c bioconda modeltest-ng -q

```

- modeltest-ng

The following packages will be downloaded:

package	build		
ca-certificates-2026.4.22	hbd8a1cb_0	128 KB	conda-forge
certifi-2026.4.22	pyhd8ed1ab_0	132 KB	conda-forge
conda-24.11.3	py311h38be061_0	1.1 MB	conda-forge
libgfortran-14.2.0	h69a702a_2	52 KB	conda-forge
libgfortran-ng-14.2.0	h69a702a_2	53 KB	conda-forge
libgfortran5-14.2.0	hf1ad2bd_2	1.4 MB	conda-forge
modeltest-ng-0.1.7	hf316886_3	5.8 MB	bioconda
mpi-1.0	openmpi	4 KB	conda-forge
openmpi-4.1.6	hc5af2df_101	3.9 MB	conda-forge
openssl-3.6.2	h35e630c_0	3.0 MB	conda-forge
zlib-1.3.1	hb9d3cd8_2	90 KB	conda-forge
Total:		15.7 MB	

The following NEW packages will be INSTALLED:

libgfortran	conda-forge/linux-64::libgfortran-14.2.0-h69a702a_2
libgfortran-ng	conda-forge/linux-64::libgfortran-ng-14.2.0-h69a702a_2
libgfortran5	conda-forge/linux-64::libgfortran5-14.2.0-hf1ad2bd_2
modeltest-ng	bioconda/linux-64::modeltest-ng-0.1.7-hf316886_3
mpi	conda-forge/linux-64::mpi-1.0-openmpi
openmpi	conda-forge/linux-64::openmpi-4.1.6-hc5af2df_101
zlib	conda-forge/linux-64::zlib-1.3.1-hb9d3cd8_2

The following packages will be UPDATED:

ca-certificates	conda-forge/linux-64::ca-certificates~	-->	conda-forge/no
certifi	2024.12.14-pyhd8ed1ab_0	-->	2026.4.22-pyhd
conda	24.11.2-py311h38be061_1	-->	24.11.3-py311h
openssl	3.4.0-h7b32b05_1	-->	3.6.2-h35e630c

done

```
# =====
# STEP 0C – Verify installations
# =====
!which raxml-ng && raxml-ng --version 2>&1 | head -3
!which modeltest-ng && modeltest-ng --version 2>&1 | head -3
```

```
/usr/local/bin/raxml-ng
```

```
RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
/usr/local/bin/modeltest-ng
```

```

      | | | | | | | | | |
_ _ _ _ _ | | _ | | _ _ _ | | | \ | / _ _ |

```

```
# =====
# STEP 1 – Retrieve Both Query Sequences from NCBI
# =====
from Bio import SeqIO, Entrez
import time, os

Entrez.email = "your.email@example.com" # <-- Replace with your real email

queries = {
    "KIF5B_Hs_Kinesin1": "NP_004512", # Human KIF5B – Kinesin-1 (plus-end)
    "KIFC1_Hs_Kinesin14": "NP_002254", # Human KIFC1 – Kinesin-14 / HSET (mi
}

os.makedirs("data", exist_ok=True)
query_records = {}

for name, acc in queries.items():
    handle = Entrez.efetch(db="protein", rettype="fasta", id=acc)
    rec = SeqIO.read(handle, "fasta")
    handle.close()
    rec.id = name
    rec.description = ""
    query_records[name] = rec
    fasta_path = f"data/{name}.fasta"
    SeqIO.write(rec, fasta_path, "fasta")
    print(f"[{name}] length={len(rec.seq)} saved → {fasta_path}")
    time.sleep(0.5)
```

```
[KIF5B_Hs_Kinesin1] length=963 saved → data/KIF5B_Hs_Kinesin1.fasta
[KIFC1_Hs_Kinesin14] length=673 saved → data/KIFC1_Hs_Kinesin14.fasta
```

```
# =====
# STEP 2A – BLAST: Kinesin Motor (KIF5B) vs PDB
```

```
# =====
# NOTE: Takes ~3-5 minutes
%%time
from Bio.Blast import NCBIWWW

result_kinesin = NCBIWWW.qblast(
    "blastp", "pdb",
    query_records["KIF5B_Hs_Kinesin1"].seq,
    hitlist_size=30
)
with open("data/blast_kinesin_motor.xml", "w") as f:
    f.write(result_kinesin.read())
result_kinesin.close()
print("Done → data/blast_kinesin_motor.xml")
```

Done → data/blast_kinesin_motor.xml
 CPU times: user 24.1 ms, sys: 5.7 ms, total: 29.8 ms
 Wall time: 1min

```
# =====
# STEP 2B – BLAST: Kinesin-14 (KIFC1) vs PDB
# =====
# NOTE: Takes ~3-5 minutes
%%time
from Bio.Blast import NCBIWWW

result_k14 = NCBIWWW.qblast(
    "blastp", "pdb",
    query_records["KIFC1_Hs_Kinesin14"].seq,
    hitlist_size=30
)
with open("data/blast_kinesin14.xml", "w") as f:
    f.write(result_k14.read())
result_k14.close()
print("Done → data/blast_kinesin14.xml")
```

Done → data/blast_kinesin14.xml
 CPU times: user 17.9 ms, sys: 4.1 ms, total: 22 ms
 Wall time: 1min 16s

```
# =====
# STEP 3 – Parse Both BLAST XMLs
# =====
from Bio.Blast import NCBIXML

with open("data/blast_kinesin_motor.xml") as f:
    blast_kinesin = NCBIXML.read(f)
with open("data/blast_kinesin14.xml") as f:
    blast_k14 = NCBIXML.read(f)
```

```
print(f"Kinesin motor BLAST hits : {len(blast_kinesin.alignments)}")
print(f"Kinesin-14 BLAST hits : {len(blast_k14.alignments)}")
```

Kinesin motor BLAST hits : 30

Kinesin-14 BLAST hits : 30

```
# =====
# STEP 4 – Display BLAST Hits
# =====
def print_hits(blast_record, label):
    print(f"\n{label}")
    print(f"{'Hit ID':<28} {'Aln Len':>8} {'E-value':>12} {'PID%':>7}")
    print("-" * 60)
    for aln in blast_record.alignments:
        for hsp in aln.hsps:
            pid = 100 * hsp.identities / hsp.align_length
            print(f"{aln.hit_id:<28} {hsp.align_length:>8} "
                  f"{hsp.expect:>12.2e} {pid:>7.1f}")

print_hits(blast_kinesin, "KINESIN MOTOR (KIF5B) hits")
print_hits(blast_k14, "KINESIN-14 (KIFC1) hits")
```

KINESIN MOTOR (KIF5B) hits

Hit ID	Aln Len	E-value	PID%
-----	-----	-----	-----
pdb 8RHB K	963	0.00e+00	100.0
pdb 9GNQ K	357	0.00e+00	100.0
pdb 9GNQ K	24	1.14e-02	70.8
pdb 1MKJ A	349	0.00e+00	100.0
pdb 8IXA S	349	0.00e+00	99.7
pdb 9L7M K	349	0.00e+00	99.4
pdb 3J8X K	349	0.00e+00	98.9
pdb 4HNA K	349	0.00e+00	98.3
pdb 9L7E A	349	0.00e+00	98.0
pdb 4ATX C	340	0.00e+00	98.8
pdb 1BG2 A	325	0.00e+00	100.0
pdb 9L78 A	335	0.00e+00	97.9
pdb 9L6K A	335	0.00e+00	97.9
pdb 4LNU K	325	0.00e+00	98.5
pdb 5LT3 A	325	0.00e+00	98.2
pdb 5LT1 A	325	0.00e+00	98.2
pdb 60JQ K	317	0.00e+00	98.7
pdb 3J6H K	342	0.00e+00	86.8
pdb 3WRD A	334	0.00e+00	87.1
pdb 4UXT C	334	0.00e+00	85.3
pdb 9UMM A	328	0.00e+00	85.4
pdb 5HNY K	321	0.00e+00	87.2
pdb 5HLE A	321	0.00e+00	85.7
pdb 2Y5W A	355	0.00e+00	75.8
pdb 2KIN A	238	3.86e-148	85.3
pdb 1G0J A	347	8.09e-126	55.6
pdb 1IA0 K	372	3.91e-84	42.5
pdb 3B6U A	342	2.61e-80	45.3

pdb 1I5S A	363	3.71e-79	42.1
pdb 1VFV A	363	4.69e-79	42.1
pdb 1I6I A	363	4.94e-79	42.1

KINESIN-14 (KIFC1) hits

Hit ID	Aln Len	E-value	PID%
-----	-----	-----	-----
pdb 5WDH A	357	0.00e+00	99.7
pdb 2NCD A	433	5.50e-97	41.8
pdb 4GKR A	378	1.19e-95	44.7
pdb 5W3D A	421	1.91e-95	42.5
pdb 1CZ7 A	413	7.22e-95	42.4
pdb 3U06 A	421	1.54e-94	42.3
pdb 1N6M A	421	1.82e-94	42.3
pdb 8YUE A	421	3.55e-94	42.3
pdb 8YY2 C	421	5.50e-94	42.3
pdb 4ETP A	412	6.36e-94	41.5
pdb 1F9T A	379	7.31e-94	44.6
pdb 3L1C A	412	1.92e-93	42.5
pdb 3T0Q A	371	1.23e-90	45.8
pdb 3KAR A	367	2.17e-86	43.6
pdb 1F9W A	369	5.63e-86	43.1
pdb 1F9U A	369	6.53e-86	43.1
pdb 1F9V A	369	7.58e-86	43.1
pdb 4H1G A	364	1.15e-76	42.3
pdb 5WDE A	353	3.73e-75	43.3

```
# =====
# STEP 5 – Filter Hits by Percent Identity
# =====
PIDcut = 1.00 # accept everything ≤ 100 %

def filter_hits(blast_record, query_len, label):
    print(f"\nFiltered hits: {label}")
    print(f"{'Accession':<28} {'SeqLen':>7} {'AlnLen':>7} "
          f"{'E-val':>10} {'PID%':>6} {'Cov%':>6}")
    print("-" * 70)
    passed = []
    for aln in blast_record.alignments:
        for hsp in aln.hsps:
            pid = hsp.identities / hsp.align_length
            if pid <= PIDcut:
                cov = 100 * sum(c.isalpha() for c in hsp.query) / query_len
                print(f"{'aln.hit_id':<28} {'aln.length':>7} {'hsp.align_length':>7} "
                      f"{'hsp.expect':>10.2e} {'100*pid':>6.1f} {'cov':>6.1f}")
                passed.append(aln.hit_id)
    print(f" {len(passed)} hits passed filter")
    return passed

kinesin_hits = filter_hits(blast_kinesin, len(query_records["KIF5B_Hs_Kinesin1
kinesin14_hits = filter_hits(blast_k14, len(query_records["KIFC1_Hs_Kinesin1
```

pdb 9L78 A	342	335	0.00e+00	97.9	34.8
pdb 9L6K A	342	335	0.00e+00	97.9	34.8
pdb 4LNU K	325	325	0.00e+00	98.5	33.7
pdb 5LT3 A	325	325	0.00e+00	98.2	33.7
pdb 5LT1 A	325	325	0.00e+00	98.2	33.7
pdb 6OJQ K	317	317	0.00e+00	98.7	32.9
pdb 3J6H K	352	342	0.00e+00	86.8	35.3
pdb 3WRD A	341	334	0.00e+00	87.1	34.5
pdb 4UXT C	340	334	0.00e+00	85.3	34.6
pdb 9UMM A	334	328	0.00e+00	85.4	34.0
pdb 5HNY K	371	321	0.00e+00	87.2	33.3
pdb 5HLE A	371	321	0.00e+00	85.7	33.3
pdb 2Y5W A	365	355	0.00e+00	75.8	36.4
pdb 2KIN A	238	238	3.86e-148	85.3	24.6
pdb 1G0J A	355	347	8.09e-126	55.6	35.4
pdb 1IA0 K	394	372	3.91e-84	42.5	35.1
pdb 3B6U A	372	342	2.61e-80	45.3	33.7
pdb 1I5S A	367	363	3.71e-79	42.1	34.2
pdb 1VFV A	366	363	4.69e-79	42.1	34.2
pdb 1I6I A	366	363	4.94e-79	42.1	34.2

31 hits passed filter

Filtered hits: Kinesin-14

Accession	SeqLen	AlnLen	E-val	PID%	Cov%
-----	-----	-----	-----	-----	-----
pdb 5WDH A	376	357	0.00e+00	99.7	53.0
pdb 2NCD A	420	433	5.50e-97	41.8	63.3
pdb 4GKR A	371	378	1.19e-95	44.7	54.4
pdb 5W3D A	412	421	1.91e-95	42.5	61.5
pdb 1CZ7 A	406	413	7.22e-95	42.4	60.3
pdb 3U06 A	412	421	1.54e-94	42.3	61.5
pdb 1N6M A	409	421	1.82e-94	42.3	61.5
pdb 8YUE A	390	421	3.55e-94	42.3	61.5
pdb 8YY2 C	409	421	5.50e-94	42.3	61.5
pdb 4ETP A	403	412	6.36e-94	41.5	59.7
pdb 1F9T A	358	379	7.31e-94	44.6	54.2
pdb 3L1C A	383	412	1.92e-93	42.5	60.2
pdb 3T0Q A	349	371	1.23e-90	45.8	53.5
pdb 3KAR A	346	367	2.17e-86	43.6	52.5
pdb 1F9W A	347	369	5.63e-86	43.1	52.7
pdb 1F9U A	347	369	6.53e-86	43.1	52.7
pdb 1F9V A	347	369	7.58e-86	43.1	52.7
pdb 4H1G A	715	364	1.15e-76	42.3	53.3
pdb 5WDE A	330	353	3.73e-75	43.3	52.5
pdb 3H4S A	386	368	1.35e-74	39.7	54.7
pdb 1SDM A	369	364	4.08e-71	40.1	53.5
pdb 5HLE A	371	387	1.69e-63	37.7	57.4
pdb 5HNY K	371	387	3.51e-61	37.0	57.4
pdb 3B6U A	372	316	3.92e-58	41.8	45.5
pdb 6A20 A	437	350	6.42e-57	38.6	47.8
pdb 6A1Z A	395	349	1.13e-56	38.7	47.7
pdb 7D8V A	437	350	1.93e-56	38.6	47.8
pdb 5ZBS A	375	349	1.96e-56	38.7	47.7
pdb 5ZBR A	375	349	2.12e-56	38.7	47.7
pdb 3GBJ A	354	339	1.04e-55	39.2	46.8

30 hits passed filter

```

# =====
# STEP 6 – Download Top Hits & Build Combined FASTA
# =====
from Bio import SeqIO, Entrez
import time

Entrez.email = "your.email@example.com" # <-- Replace with your real email

def fetch_hits(blast_record, top_n, label, pid_cut=1.00):
    records = []
    seen_ids = set()
    for aln in blast_record.alignments[:top_n]:
        for hsp in aln.hsps:
            if aln.hit_id in seen_ids:
                continue
            pid = hsp.identities / hsp.align_length
            if pid <= pid_cut:
                print(f" Fetching [{label}]: {aln.hit_id} ... ", end="")
                try:
                    handle = Entrez.efetch(db="protein", id=aln.hit_id, rettype="fasta")
                    rec = SeqIO.read(handle, "fasta")
                    handle.close()
                    records.append(rec)
                    seen_ids.add(aln.hit_id)
                    print("OK")
                except Exception as e:
                    print(f"FAILED ({e})")
                time.sleep(0.4)
    print(f" Fetched {len(records)} sequences for {label}")
    return records

print("Fetching Kinesin motor hits...")
kinesin_seqs = fetch_hits(blast_kinesin, top_n=15, label="KinesinMotor")
print("\nFetching Kinesin-14 hits...")
kinesin14_seqs = fetch_hits(blast_k14, top_n=15, label="Kinesin14")

# Merge: query sequences first, then BLAST hits (deduplicated by sequence)
all_records = list(query_records.values())
seen_seqs = {str(r.seq).upper() for r in all_records}

for rec in kinesin_seqs + kinesin14_seqs:
    seq_str = str(rec.seq).upper()
    if seq_str not in seen_seqs:
        all_records.append(rec)
        seen_seqs.add(seq_str)

print(f"\nTotal unique sequences: {len(all_records)}")
SeqIO.write(all_records, "data/sequences.fasta", "fasta")
print("Saved → data/sequences.fasta")

```

```

Fetching Kinesin motor hits...
  Fetching [KinesinMotor]: pdb|8RHB|K ... OK
  Fetching [KinesinMotor]: pdb|9GNQ|K ... OK
  Fetching [KinesinMotor]: pdb|1MKJ|A ... OK
  Fetching [KinesinMotor]: pdb|8IXA|S ... OK
  Fetching [KinesinMotor]: pdb|9L7M|K ... OK
  Fetching [KinesinMotor]: pdb|3J8X|K ... OK
  Fetching [KinesinMotor]: pdb|4HNA|K ... OK
  Fetching [KinesinMotor]: pdb|9L7E|A ... OK
  Fetching [KinesinMotor]: pdb|4ATX|C ... OK
  Fetching [KinesinMotor]: pdb|1BG2|A ... OK
  Fetching [KinesinMotor]: pdb|9L78|A ... OK
  Fetching [KinesinMotor]: pdb|9L6K|A ... OK
  Fetching [KinesinMotor]: pdb|4LNU|K ... OK
  Fetching [KinesinMotor]: pdb|5LT3|A ... OK
  Fetching [KinesinMotor]: pdb|5LT1|A ... OK
  Fetched 15 sequences for KinesinMotor

```

```

Fetching Kinesin-14 hits...
  Fetching [Kinesin14]: pdb|5WDH|A ... OK
  Fetching [Kinesin14]: pdb|2NCD|A ... OK
  Fetching [Kinesin14]: pdb|4GKR|A ... OK
  Fetching [Kinesin14]: pdb|5W3D|A ... OK
  Fetching [Kinesin14]: pdb|1CZ7|A ... OK
  Fetching [Kinesin14]: pdb|3U06|A ... OK
  Fetching [Kinesin14]: pdb|1N6M|A ... OK
  Fetching [Kinesin14]: pdb|8YUE|A ... OK
  Fetching [Kinesin14]: pdb|8YY2|C ... OK
  Fetching [Kinesin14]: pdb|4ETP|A ... OK
  Fetching [Kinesin14]: pdb|1F9T|A ... OK
  Fetching [Kinesin14]: pdb|3L1C|A ... OK
  Fetching [Kinesin14]: pdb|3T0Q|A ... OK
  Fetching [Kinesin14]: pdb|3KAR|A ... OK
  Fetching [Kinesin14]: pdb|1F9W|A ... OK
  Fetched 15 sequences for Kinesin14

```

```

Total unique sequences: 31
Saved → data/sequences.fasta

```

```

# =====
# STEP 7 – Multiple Sequence Alignment with MAFFT
# =====
!mafft --auto data/sequences.fasta > data/aligned.fasta

from Bio import AlignIO
aln = AlignIO.read("data/aligned.fasta", "fasta")
print(f"Alignment: {len(aln)} sequences × {aln.get_alignment_length()} columns")

All-to-all alignment.

```

```

Loading hat3.seed ...
done.
Writing hat3 for iterative refinement
rescale = 1
Gap Penalty = -1.53, +0.00, +0.00
tbtree = 1, compacttree = 0
Constructing a UPGMA tree ...
    20 / 31
done.

Progressive alignment ...
STEP    30 /30
done.
tbfast (aa) Version 7.490
alg=A, model=BLOSUM62, 1.53, -0.00, -0.00, noshift, amax=0.0
1 thread(s)

minimumweight = 0.000010
autosubalignment = 0.000000
nthread = 0
randomseed = 0
blosum 62 / kimura 200
poffset = 0
niter = 16
sueff_global = 0.100000
nadd = 16
Loading 'hat3' ... done.
rescale = 1

    20 / 31
Segment 1/ 1    1-1325
STEP 003-015-0 identical.
Converged.

done
dvtidtr (aa) Version 7.490
alg=A, model=BLOSUM62, 1.53, -0.00, -0.00, noshift, amax=0.0
0 thread(s)

Strategy:
  L-INS-i (Probably most accurate, very slow)
  Iterative refinement method (<16) with LOCAL pairwise alignment information

If unsure which option to use, try 'mafft --auto input > output'.
For more information, see 'mafft --help', 'mafft --man' and the mafft page.

The default gap scoring scheme has been changed in version 7.110 (2013 Oct).
It tends to insert more gaps into gap-rich regions than previous versions.
To disable this change, add the --leavegappyregion option.

Alignment: 31 sequences x 1324 columns

```

```

# =====
# STEP 8 – Trim Gappy Terminal Columns
# =====
from Bio.Align import SeqIO

```

```
from Bio import AlignIO, SeqIO
```

```
aln = AlignIO.read("data/aligned.fasta", "fasta")
aln_len = aln.get_alignment_length()

position1 = next(
    col for col in range(aln_len) if "-" not in aln[:, col]
)
position2 = next(
    col + 1 for col in reversed(range(aln_len)) if "-" not in aln[:, col]
)

print(f"Original length : {aln_len} columns")
print(f"Keeping columns : {position1} → {position2}")
print(f"Trimmed length : {position2 - position1} columns")

with open("data/aligned_trimmed.fasta", "w") as fh:
    SeqIO.write(aln[:, position1:position2], fh, "fasta")

trim = AlignIO.read("data/aligned_trimmed.fasta", "fasta")
print(f"Trimmed alignment: {len(trim)} sequences × {trim.get_alignment_length()}")
```

```
Original length : 1324 columns
Keeping columns : 314 → 685
Trimmed length : 371 columns
Trimmed alignment: 31 sequences × 371 columns
```

```
# =====
# STEP 9 – Pairwise Distance Matrix (identity model)
# =====
from Bio.Phylo.TreeConstruction import DistanceCalculator

calculator = DistanceCalculator("identity")
dm = calculator.get_distance(trim)
print("Pairwise distance matrix (identity model):\n")
print(dm)
```

Pairwise distance matrix (identity model):

KIF5B_Hs_Kinesin1	0.000000					
KIFC1_Hs_Kinesin14	0.654987	0.000000				
pdb 9GNQ K	0.000000	0.654987	0.000000			
pdb 1MKJ A	0.000000	0.654987	0.000000	0.000000		
pdb 8IXA S	0.002695	0.657682	0.002695	0.002695	0.000000	
pdb 9L7M K	0.000000	0.654987	0.000000	0.000000	0.002695	0.000000
pdb 3J8X K	0.008086	0.649596	0.008086	0.008086	0.010782	0.008086
pdb 4HNA K	0.013477	0.646900	0.013477	0.013477	0.016173	0.013477
pdb 9L7E A	0.016173	0.649596	0.016173	0.016173	0.018868	0.016173
pdb 4ATX C	0.008086	0.657682	0.008086	0.008086	0.010782	0.008086
pdb 1BG2 A	0.000000	0.654987	0.000000	0.000000	0.002695	0.000000
pdb 9L78 A	0.016173	0.649596	0.016173	0.016173	0.018868	0.016173
pdb 9L6K A	0.016173	0.649596	0.016173	0.016173	0.018868	0.016173
pdb 4LNU K	0.013477	0.646900	0.013477	0.013477	0.016173	0.013477

pdb 5LT3 A	0.016173	0.649596	0.016173	0.016173	0.018868	0.016173
pdb 5LT1 A	0.016173	0.649596	0.016173	0.016173	0.018868	0.016173
pdb 5WDH A	0.654987	0.002695	0.654987	0.654987	0.657682	0.654987
pdb 2NCD A	0.582210	0.563342	0.582210	0.582210	0.584906	0.582210
pdb 4GKR A	0.638814	0.568733	0.638814	0.638814	0.641509	0.638814
pdb 5W3D A	0.582210	0.563342	0.582210	0.582210	0.584906	0.582210
pdb 1CZ7 A	0.582210	0.563342	0.582210	0.582210	0.584906	0.582210
pdb 3U06 A	0.582210	0.566038	0.582210	0.582210	0.584906	0.582210
pdb 1N6M A	0.584906	0.566038	0.584906	0.584906	0.587601	0.584906
pdb 8YUE A	0.584906	0.566038	0.584906	0.584906	0.587601	0.584906
pdb 8YY2 C	0.584906	0.566038	0.584906	0.584906	0.587601	0.584906
pdb 4ETP A	0.628032	0.571429	0.628032	0.628032	0.630728	0.628032
pdb 1F9T A	0.628032	0.571429	0.628032	0.628032	0.630728	0.628032
pdb 3L1C A	0.584906	0.566038	0.584906	0.584906	0.587601	0.584906
pdb 3T0Q A	0.633423	0.560647	0.633423	0.633423	0.636119	0.633423
pdb 3KAR A	0.628032	0.571429	0.628032	0.628032	0.630728	0.628032
pdb 1F9W A	0.630728	0.574124	0.630728	0.630728	0.628032	0.630728
KIF5B_Hs_Kinesin1		KIFC1_Hs_Kinesin14	pdb 9GNQ K	pdb 1MKJ A	pdb 8IXA S	

```
# =====
# STEP 10 – Remove Duplicate Sequences
# =====
from Bio import SeqIO
import os

def sequence_cleaner(fasta_file, min_length=0):
    sequences = {}
    for rec in SeqIO.parse(fasta_file, "fasta"):
        seq = str(rec.seq).upper()
        if len(seq) >= min_length:
            if seq not in sequences:
                sequences[seq] = rec.id
            else:
                sequences[seq] += "_" + rec.id

    out_file = "data/clear_" + os.path.basename(fasta_file)
    with open(out_file, "w") as fh:
        for seq, seq_id in sequences.items():
            fh.write(">" + seq_id + "\n" + seq + "\n")

    n_in = sum(1 for _ in SeqIO.parse(fasta_file, "fasta"))
    n_out = len(sequences)
    print(f"Input : {n_in} sequences")
    print(f"Output: {n_out} sequences (removed {n_in - n_out} duplicates)")
    print(f"Saved → {out_file}")
    return out_file

cleaned_file = sequence_cleaner("data/aligned_trimmed.fasta", min_length=0)
```

```
Input : 31 sequences
Output: 22 sequences (removed 9 duplicates)
Saved → data/clear_aligned_trimmed.fasta
```

```
# =====
# STEP 11 – Model Selection with ModelTest-NG
# =====
import subprocess, re

modeltest_available = subprocess.run(
    ["which", "modeltest-ng"], capture_output=True
).returncode == 0

if modeltest_available:
    result = subprocess.run(
        ["modeltest-ng", "-i", cleaned_file, "-d", "aa"],
        capture_output=True, text=True
    )
    print(result.stdout[-3000:]) # show tail of output

    # Parse best BIC model from stdout
    match = re.search(r"BIC\s+\|\s+(\S+)", result.stdout)
    MODEL = match.group(1) if match else "LG+I+G4"
else:
    print("ModelTest-NG not found – using default model LG+I+G4")
    MODEL = "LG+I+G4"

print(f"\n>>> Selected substitution model: {MODEL}")
```

```
-----
1  LG+G4          1  -4102.0754      8299.1508      0.0000
2  LG+I+G4        2  -4101.8754      8300.7508      1.6001
3  VT+I+G4        2  -4106.3254      8309.6508     10.5000
4  VT+G4          1  -4107.7342      8310.4684     11.3177
5  LG+G4+F       20  -4082.8443      8311.6886     12.5379
6  LG+I+G4+F     21  -4082.8483      8314.6965     15.5458
7  RTREV+G4+F    20  -4086.3815      8318.7629     19.6122
8  RTREV+I+G4+F  21  -4086.2348      8321.4697     22.3189
9  WAG+I+G4      2  -4113.1589      8323.3178     24.1671
10 WAG+G4        1  -4114.2779      8323.5558     24.4051
-----
```

Best model according to AICc

```
-----
Model:          LG+G4
lnL:            -4102.0754
Frequencies:    0.0791 0.0559 0.0420 0.0531 0.0129 0.0408 0.0716 0.0573 0.0
Inv. sites prop: -
Gamma shape:    0.7301
Score:          8299.1508
Weight:         0.6847
-----
```

Frequencies: 0.0016

Model averaged estimates

P.Inv: 0.1786

Alpha: 0.7308

Alpha-P.Inv: 0.9337

P.Inv-Alpha: 0.0597

Frequencies: 0.0478 0.0507 0.0663 0.0529 0.0152 0.0382 0.0686 0.0617 0.0

Commands:

```
> phyml -i data/clear_aligned_trimmed.fasta -d aa -m LG -f m -v 0 -a e -c 4
> raxmlHPC-SSE3 -s data/clear_aligned_trimmed.fasta -m PROTGAMMALG -n EXEC_N
> raxml-ng --msa data/clear_aligned_trimmed.fasta --model LG+G4
> paup -s data/clear_aligned_trimmed.fasta
> iqtree -s data/clear_aligned_trimmed.fasta -m LG+G4
```

Summary:

Partition 1/1:

	Model	Score	Weight
BIC	LG+G4	8452.6312	0.9366
AIC	LG+G4+F	8287.6886	0.3898
AICc	LG+G4	8299.1508	0.6847

Execution results written to data/clear_aligned_trimmed.fasta.out

Starting tree written to data/clear_aligned_trimmed.fasta.tree

>>> Selected substitution model: LG+I+G4

```
# =====
# STEP 12 – Build ML Tree with RAXML-NG
# =====
!rm -f T1.raxml.*
!raxml-ng \
  --msa      {cleaned_file} \
  --model    {MODEL}        \
  --prefix   T1              \
  --threads  2               \
  --seed     12345

if os.path.exists("T1.raxml.bestTree"):
    print("\nBest ML tree found: T1.raxml.bestTree")
    with open("T1.raxml.bestTree") as f:
        print(f.read()[:500])
else:
    print("ERROR: Tree not produced – check RAXML-NG output above.")
```

[00:01:35] [worker #0] ML tree search #17, logLikelihood: -4101.600772

[00:01:35 -5438.294207] Initial branch length optimization

[00:01:35] [worker #1] ML tree search #18, logLikelihood: -4101.591557

[00:01:35 -4109.575664] Model parameter optimization (eps = 10.000000)

[00:01:35 -4106.481614] AUTODETECT spr round 1 (radius: 5)

[00:01:36 -4101.887100] AUTODETECT spr round 2 (radius: 10)

[00:01:36 -4101.834553] SPR radius for FAST iterations: 5 (autodetect)

[00:01:36 -4101.834553] Model parameter optimization (eps = 3.000000)

[00:01:36 -4101.668772] FAST spr round 1 (radius: 5)

[00:01:37 -4101.666819] Model parameter optimization (eps = 1.000000)

[00:01:37 -4101.622522] SLOW spr round 1 (radius: 5)

[00:01:40 -4101.622493] SLOW spr round 2 (radius: 10)

[00:01:42 -4101.622493] SLOW spr round 3 (radius: 15)

[00:01:42 -4101.622493] SLOW spr round 4 (radius: 20)

[00:01:42 -4101.622493] Model parameter optimization (eps = 0.100000)

[00:01:42] [worker #0] ML tree search #19, logLikelihood: -4101.588610

[00:01:42] [worker #1] ML tree search #20, logLikelihood: -4101.594544

Optimized model parameters:

Partition 0: noname

Rate heterogeneity: GAMMA (4 cats, mean), alpha: 1.019533 (ML), weights & P-inv (ML): 0.075596

Base frequencies (model): 0.079066 0.055941 0.041977 0.053052 0.012937 0.041977

Substitution rates (model): 0.425093 0.276818 0.395144 2.489084 0.969894 1.000000

Final LogLikelihood: -4101.567387

AIC score: 8289.134775 / AICc score: 8300.706640 / BIC score: 8457.531463

Free parameters (model + branch lengths): 43

WARNING: Best ML tree contains 9 near-zero branches!

Best ML tree with collapsed near-zero branches saved to: /content/T1.raxml.best

Best ML tree saved to: /content/T1.raxml.bestTree

All ML trees saved to: /content/T1.raxml.mlTrees

Optimized model saved to: /content/T1.raxml.bestModel

Execution log saved to: /content/T1.raxml.log

Analysis started: 02-May-2026 19:33:52 / finished: 02-May-2026 19:35:35

Elapsed time: 102.702 seconds

Best ML tree found: T1.raxml.bestTree

((pdb|3T0Q|A:0.332906,(pdb|4GKR|A:0.220990,(pdb|4ETP|A:0.013896,((pdb|3KAR|A:0

=====

STEP 13 – Visualize Initial ML Tree (no bootstrap)

```
# =====
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from Bio import Phylo

COLORS = {
    "KIF5B_Hs_Kinesin1": "#2196F3",
    "KIFC1_Hs_Kinesin14": "#E53935",
}

tree = Phylo.read("T1.raxml.bestTree", "newick")
tree.ladderize()

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.
Phylo.draw(tree, axes=ax, do_show=False)

for text in ax.texts:
    for key, color in COLORS.items():
        if key in text.get_text():
            text.set_color(color)
            text.set_fontweight("bold")

ax.set_title(
    "Kinesin Motor + Kinesin-14 Combined – Best ML Tree (no bootstrap)\n"
    f"Model: {MODEL} | Blue = KIF5B (Kinesin-1) | Red = KIFC1 (Kinesin-14)"
    fontsize=12, pad=12
)
ax.legend(handles=[
    mpatches.Patch(color="#2196F3", label="Kinesin motor (Kinesin-1, KIF5B) hit
    mpatches.Patch(color="#E53935", label="Kinesin-14 (KIFC1/HSET) hits"),
], loc="lower right", fontsize=9)

plt.tight_layout()
plt.savefig("data/tree_ML.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_ML.png")
```

Saved → data/tree_ML.png

```
# =====
# STEP 14 – Bootstrap Analysis (100 replicates)
# =====
# NOTE: Takes several minutes
!rm -f T2.raxml.*
!raxml-ng \
    --bootstrap \
    --msa T1.raxml.rba \
    --model {MODEL} \
    --prefix T2 \
```

```
--threads 2      \  
--seed 12345     \  
--bs-trees 100
```

```
print("Bootstrap trees → T2.raxml.bootstraps")
```

Execution log saved to: /content/T2.raxml.log

Analysis started: 02-May-2026 19:35:58 / finished: 02-May-2026 19:40:49

Elapsed time: 290.480 seconds

Bootstrap trees → T2.raxml.bootstraps

```
# =====
# STEP 15 – Map Bootstrap Values onto Best ML Tree
# =====
!rm -f T3.raxml.*
!raxml-ng \
  --support \
  --tree T1.raxml.bestTree \
  --bs-trees T2.raxml.bootstraps \
  --prefix T3 \
  --threads 2
```

print("Bootstrap-annotated tree → T3.raxml.support")

RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.

Developed by: Alexey M. Kozlov and Alexandros Stamatakis.

Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Be

Latest version: <https://github.com/amkozlov/raxml-ng>

Questions/problems/suggestions? Please visit: <https://groups.google.com/forum/#!>

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAxML-NG was called at 02-May-2026 19:41:14 as follows:

`raxml-ng --support --tree T1.raxml.bestTree --bs-trees T2.raxml.bootstraps --pre`

Analysis options:

run mode: Compute bipartition support (Felsenstein Bootstrap)

start tree(s): user

random seed: 1777750874

SIMD kernels: AVX2

parallelization: coarse-grained (auto), PTHREADS (2 threads), thread pinning:

Reading reference tree from file: T1.raxml.bestTree

Reference tree size: 22

Reading bootstrap trees from file: T2.raxml.bootstraps

Loaded 100 trees with 22 taxa.

Best ML tree with Felsenstein bootstrap (FBP) support values saved to: /content/

Execution log saved to: /content/T3.raxml.log

Analysis started: 02-May-2026 19:41:14 / finished: 02-May-2026 19:41:14

Elapsed time: 0.006 seconds

Bootstrap-annotated tree → T3.raxml.support

```
# =====
# STEP 16 – Visualize Bootstrap-Supported Tree
# =====
from Bio import Phylo
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

tree = Phylo.read("T3.raxml.support", "newick")
tree.ladderize()

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.3)
Phylo.draw(tree, axes=ax, do_show=False)

for text in ax.texts:
    if "KIF5B_Hs_Kinesin1" in text.get_text():
        text.set_color("#2196F3"); text.set_fontweight("bold")
    elif "KIF14_Hs_Kinesin14" in text.get_text():
        text.set_color("#E53935"); text.set_fontweight("bold")

ax.set_title(
    "Kinesin Motor + Kinesin-14 – ML Tree with 100 Bootstrap Support\n"
    f"Model: {MODEL} | Node labels = bootstrap % (100 replicates)",
    fontsize=12, pad=12
)
ax.legend(handles=[
    mpatches.Patch(color="#2196F3", label="Kinesin motor (Kinesin-1) query & hit"),
    mpatches.Patch(color="#E53935", label="Kinesin-14 query & hits"),
], loc="lower right", fontsize=9)

plt.tight_layout()
plt.savefig("data/tree_bootstrap.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_bootstrap.png")
```

Saved → data/tree_bootstrap.png

```
# =====
# STEP 17 – Ancestral Sequence Reconstruction
# =====
!rm -f ASR.raxml.*
!raxml-ng \
    --ancestral \
    --msa {cleaned_file} \
    --tree T1.raxml.bestTree \
    --model {MODEL} \
    --prefix ASR
```

```
for fname in ["ASR.raxml.ancestralTree", "ASR.raxml.ancestralProbs"]:
    status = "✓ Found" if os.path.exists(fname) else "X Missing"
    print(f"{status}: {fname}")
```

RAXML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
 Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
 Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben
 Latest version: <https://github.com/amkozlov/raxml-ng>
 Questions/problems/suggestions? Please visit: <https://groups.google.com/forum/>

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAXML-NG was called at 02-May-2026 19:41:36 as follows:

```
raxml-ng --ancestral --msa data/clear_aligned_trimmed.fasta --tree T1.raxml.be
```

Analysis options:

```
run mode: Ancestral state reconstruction
start tree(s): user
random seed: 1777750896
tip-inner: ON
pattern compression: OFF
per-rate scalars: OFF
site repeats: OFF
logLH epsilon: general: 10.000000, brlen-triplet: 1000.000000
branch lengths: proportional (ML estimate, algorithm: NR-FAST)
SIMD kernels: AVX2
parallelization: coarse-grained (auto), NONE/sequential
```

```
[00:00:00] Reading alignment from file: data/clear_aligned_trimmed.fasta
[00:00:00] Loaded alignment with 22 taxa and 371 sites
```

Alignment comprises 1 partitions and 371 sites

```
Partition 0: noname
Model: LG+I+G4m
Alignment sites: 371
Gaps: 11.09 %
Invariant sites: 26.42 %
```

NOTE: Binary MSA file created: ASR.raxml.rba

Parallelization scheme autoconfig: 1 worker(s) x 1 thread(s)

```
[00:00:00] Loading user starting tree(s) from: T1.raxml.bestTree
Parallel reduction/worker buffer size: 1 KB / 0 KB
```

```
[00:00:00] Data distribution: max. partitions/sites/weight per thread: 1 / 371
[00:00:00] Data distribution: max. searches per worker: 1
```

Starting ML tree search with 1 distinct starting trees

```
[00:00:00] Tree #1, initial LogLikelihood: -4103.627686
```

```
[00:00:00 -4103.627686] Initial branch length optimization
[00:00:00 -4103.495386] Model parameter optimization (eps = 10.000000)

[00:00:00] Tree #1, final logLikelihood: -4101.897191
```

```
# =====
# STEP 18 – Visualize Ancestral Reconstruction Tree
# =====
from Bio import Phylo
import matplotlib.pyplot as plt

tree = Phylo.read("ASR.raxml.ancestralTree", "newick")
tree.ladderize()

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.5)))
Phylo.draw(tree, axes=ax, do_show=False)
ax.set_title(
    "Kinesin Motor + Kinesin-14 – Ancestral Reconstruction Tree\n"
    "Internal node labels reference ASR.raxml.ancestralProbs",
    fontsize=12, pad=12
)
plt.tight_layout()
plt.savefig("data/tree_ancestral.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_ancestral.png")
```

```
# =====
# STEP 19 – Package ALL Results into a ZIP and Download
# =====
import os, zipfile
from google.colab import files

output_files = [
    # Sequences
    "data/KIF5B_Hs_Kinesin1.fasta",
    "data/KIFC1_Hs_Kinesin14.fasta",
    "data/sequences.fasta",
    # Alignments
    "data/aligned.fasta",
    "data/aligned_trimmed.fasta",
    "data/clear_aligned_trimmed.fasta",
    # BLAST XMLs
    "data/blast_kinesin_motor.xml",
    "data/blast_kinesin14.xml",
    # RAXML-NG trees
    "T1.raxml.bestTree",
    "T1.raxml.rba",
    "T2.raxml.bootstraps",
    "T3.raxml.support",
    # ASR outputs
```

```

    "ASR.raxml.ancestralTree",
    "ASR.raxml.ancestralProbs",
    # Figures
    "data/tree_ML.png",
    "data/tree_bootstrap.png",
    "data/tree_ancestral.png",
]

zip_path = "phylogenetic_analysis_results.zip"
found, missing = [], []

with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
    for fp in output_files:
        if os.path.exists(fp):
            zf.write(fp)
            found.append(fp)
        else:
            missing.append(fp)

print(f"\n✓ {len(found)} files added to {zip_path}")
if missing:
    print(f"X {len(missing)} files not found (may not have been generated):")
    for m in missing:
        print(f"    {m}")

# Single download of the ZIP
files.download(zip_path)
print(f"\nDownload triggered: {zip_path}")

```

```

✓ 16 files added to phylogenetic_analysis_results.zip
X 1 files not found (may not have been generated):
    data/tree_ancestral.png

```

Download triggered: phylogenetic_analysis_results.zip

Start coding or [generate](#) with AI.

